# DevOps & DEPLOYMENT GUIDE

## AI-Based Monitoring Solution for Oil & Gas Industry

**Document Version**: 1.0 **Date**: December 2025 **Project Code**: OG-AI-MON-2025 **Audience**: DevOps Engineers, SREs, Platform Engineers

---

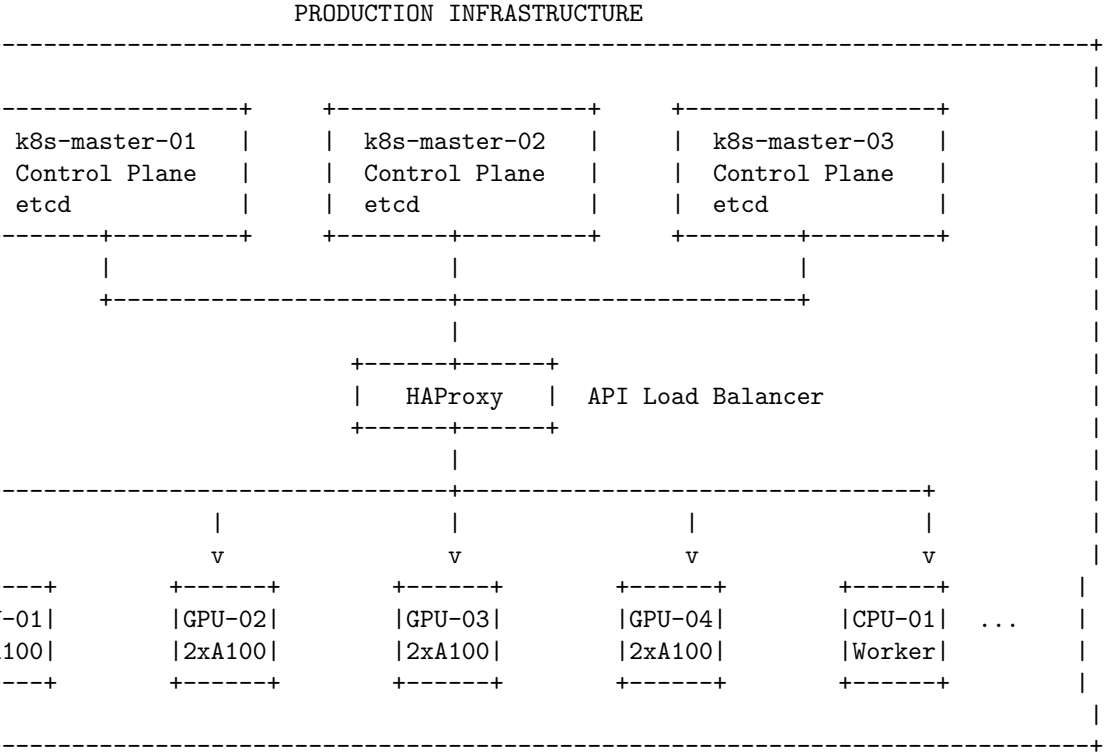
# TABLE OF CONTENTS

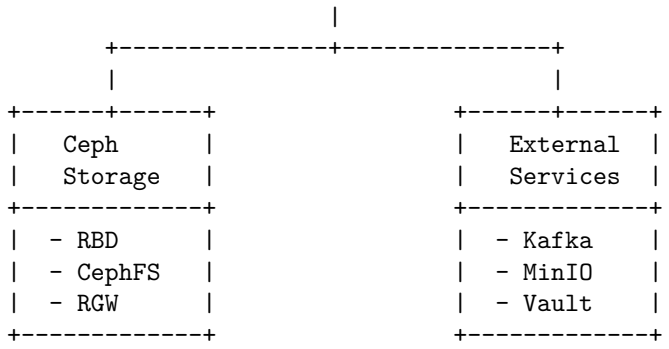

1. Infrastructure Overview
2. Kubernetes Cluster Setup
3. GitOps with ArgoCD
4. CI/CD Pipeline
5. Container Registry
6. Helm Charts
7. Secrets Management
8. Monitoring Stack
9. Logging Stack
10. Disaster Recovery
11. Edge Deployment
12. Runbooks


---

# 1. INFRASTRUCTURE OVERVIEW

## 1.1 Architecture Diagram






### 1.2 Environment Matrix

| Environment | Purpose | Nodes | Namespace | URL |
|---|---|---|---|---|
| Development | Dev testing | 3 | oilgas-dev | dev.oilgas.internal |
| QA | Testing | 6 | oilgas-qa | qa.oilgas.internal |
| Staging | Pre-prod | 6 | oilgas-staging | staging.oilgas.internal |
| Production | Live | 13 | oilgas-prod | api.oilgas.com |

### 1.3 Technology Stack

| Category | Technology | Version |
|---|---|---|
| Container Runtime | containerd | 1.7.x |
| Orchestration | Kubernetes | 1.29.x |
| Service Mesh | Istio | 1.20.x |
| GitOps | ArgoCD | 2.10.x |
| CI/CD | GitHub Actions | - |
| Registry | Harbor | 2.10.x |
| Secrets | HashiCorp Vault | 1.15.x |
| Monitoring | Prometheus/Grafana | 2.50/10.x |
| Logging | ELK Stack | 8.12.x |
| Storage | Ceph | 18.x (Reef) |

---

## 2. KUBERNETES CLUSTER SETUP

### 2.1 Cluster Initialization

**Prerequisites**

```
# All nodes: Install containerd
cat <<EOF | sudo tee /etc/modules-load.d/containerd.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter
```

```
cat <<EOF | sudo tee /etc/sysctl.d/99-kubernetes-cri.conf
net.bridge.bridge-nf-call-iptables  = 1
net.ipv4.ip_forward                 = 1
net.bridge.bridge-nf-call-ip6tables = 1
EOF

sudo sysctl --system

# Install containerd
sudo apt-get update
sudo apt-get install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo systemctl restart containerd

# Install kubeadm, kubelet, kubectl
sudo apt-get install -y apt-transport-https ca-certificates curl
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o /etc/apt/key
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1

sudo apt-get update
sudo apt-get install -y kubelet=1.29.0-1.1 kubeadm=1.29.0-1.1 kubectl=1.29.0-1.1
sudo apt-mark hold kubelet kubeadm kubectl
```

**Initialize First Master**

```
# Create kubeadm config
cat <<EOF > kubeadm-config.yaml
apiVersion: kubeadm.k8s.io/v1beta3
kind: ClusterConfiguration
kubernetesVersion: v1.29.0
controlPlaneEndpoint: "k8s-api.oilgas.internal:6443"
networking:
  podSubnet: "10.244.0.0/16"
  serviceSubnet: "10.96.0.0/12"
etcd:
  local:
    dataDir: /var/lib/etcd
apiServer:
  certSANs:
    - "k8s-api.oilgas.internal"
    - "10.0.1.10"
  extraArgs:
    audit-log-path: /var/log/kubernetes/audit.log
    audit-policy-file: /etc/kubernetes/audit-policy.yaml
---
apiVersion: kubeadm.k8s.io/v1beta3
kind: InitConfiguration
nodeRegistration:
  criSocket: unix:///var/run/containerd/containerd.sock
EOF

# Initialize cluster
sudo kubeadm init --config=kubeadm-config.yaml --upload-certs
```

```
# Setup kubectl
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Install Calico CNI
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.0/manifests/calico.yaml
```

### Join Additional Masters

```
# On master-01, get join command
kubeadm token create --print-join-command --certificate-key $(kubeadm init phase upload-certs --upload-

# On master-02, master-03
sudo kubeadm join k8s-api.oilgas.internal:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>
```

### Join Worker Nodes

```
# GPU workers need NVIDIA runtime
# Install NVIDIA Container Toolkit first
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -s -L https://nvidia.github.io/libnvidia-container/gpgkey | sudo apt-key add -
curl -s -L https://nvidia.github.io/libnvidia-container/$distribution/libnvidia-container.list | sudo t

sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=containerd
sudo systemctl restart containerd

# Join cluster
sudo kubeadm join k8s-api.oilgas.internal:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>

# Label GPU nodes
kubectl label node k8s-gpu-01 node.kubernetes.io/gpu=true
kubectl label node k8s-gpu-01 nvidia.com/gpu.present=true
```

## 2.2 NVIDIA GPU Operator

```
# gpu-operator-values.yaml
operator:
  defaultRuntime: containerd

driver:
  enabled: true
  version: "535.104.05"

toolkit:
  enabled: true
```

```yaml
devicePlugin:
  enabled: true

dcgm:
  enabled: true

dcgmExporter:
  enabled: true
  serviceMonitor:
    enabled: true

mig:
  strategy: none

gfd:
  enabled: true
```

```bash
# Install GPU Operator
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia
helm repo update

helm install gpu-operator nvidia/gpu-operator \
  --namespace gpu-operator \
  --create-namespace \
  -f gpu-operator-values.yaml

# Verify
kubectl get pods -n gpu-operator
kubectl get nodes -o json | jq '.items[] | {name:.metadata.name, gpu:.status.allocatable["nvidia.com/gpu
```

## 2.3 Storage Configuration

### Ceph CSI Driver

```yaml
# ceph-csi-values.yaml
csiConfig:
  - clusterID: "oilgas-ceph"
    monitors:
      - "10.0.3.11:6789"
      - "10.0.3.12:6789"
      - "10.0.3.13:6789"

storageClass:
  create: true
  name: ceph-rbd
  clusterID: oilgas-ceph
  pool: kubernetes
  imageFeatures: "layering"
  reclaimPolicy: Retain
  volumeBindingMode: Immediate
  allowVolumeExpansion: true
```

```bash
# Install Ceph CSI
helm repo add ceph-csi https://ceph.github.io/csi-charts
helm install ceph-csi-rbd ceph-csi/ceph-csi-rbd \
```

```
  --namespace ceph-csi \
  --create-namespace \
  -f ceph-csi-values.yaml

# Create Ceph secret
kubectl create secret generic csi-rbd-secret \
  --namespace ceph-csi \
  --from-literal=userID=kubernetes \
  --from-literal=userKey=<ceph-user-key>
```

---

# 3. GitOps with ArgoCD

## 3.1 ArgoCD Installation

```
# Install ArgoCD
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/v2.10.0/manifests/install

# Patch for HA
kubectl patch deployment argocd-server -n argocd \
  --type='json' -p='[{"op": "replace", "path": "/spec/replicas", "value": 3}]'

# Get initial admin password
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d

# Access UI
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

## 3.2 ArgoCD Configuration

```
# argocd-cm.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-cm
  namespace: argocd
data:
  url: https://argocd.oilgas.internal
  repositories: |
    - url: git@github.com:oilgas/platform-gitops.git
      sshPrivateKeySecret:
        name: github-ssh-key
        key: sshPrivateKey
  resource.customizations: |
    networking.k8s.io/Ingress:
      health.lua: |
        hs = {}
        hs.status = "Healthy"
        return hs
```

### 3.3 Application Definitions

#### Root Application (App of Apps)

```yaml
# applications/root.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root
  namespace: argocd
spec:
  project: default
  source:
    repoURL: git@github.com:oilgas/platform-gitops.git
    targetRevision: main
    path: applications
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

#### API Application

```yaml
# applications/api.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: oilgas-api
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: oilgas
  source:
    repoURL: git@github.com:oilgas/platform-gitops.git
    targetRevision: main
    path: charts/api
    helm:
      valueFiles:
        - values-production.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: oilgas-prod
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
    retry:
      limit: 3
      backoff:
```

```
        duration: 5s
        factor: 2
        maxDuration: 3m
```

### 3.4 GitOps Repository Structure

```
platform-gitops/
   applications/           # ArgoCD Application definitions
      root.yaml
      api.yaml
      frontend.yaml
      ml-inference.yaml
      rag-service.yaml
      monitoring.yaml

   charts/                 # Helm charts
      api/
         Chart.yaml
         values.yaml
         values-dev.yaml
         values-staging.yaml
         values-production.yaml
         templates/
      frontend/
      ml-inference/
      rag-service/

   base/                   # Base Kubernetes manifests
      namespaces/
      network-policies/
      resource-quotas/
      limit-ranges/

   overlays/               # Kustomize overlays
      dev/
      staging/
      production/

   scripts/                # Utility scripts
       bootstrap.sh
       sync-all.sh
```

---

## 4. CI/CD PIPELINE

### 4.1 GitHub Actions Workflow

```
# .github/workflows/build-deploy.yml
name: Build and Deploy

on:
  push:
    branches: [main, develop]
```

```yaml
    tags: ['v*']
  pull_request:
    branches: [main]

env:
  REGISTRY: harbor.oilgas.internal
  IMAGE_NAME: oilgas/api

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install -r requirements-dev.txt

      - name: Lint
        run: |
          ruff check .
          black --check .

      - name: Test
        run: pytest tests/ --cov=app --cov-report=xml

      - name: SonarQube Scan
        uses: sonarsource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}

  build:
    needs: test
    runs-on: ubuntu-latest
    outputs:
      image_tag: ${{ steps.meta.outputs.tags }}
      image_digest: ${{ steps.build.outputs.digest }}
    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to Harbor
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
```

```
        username: ${{ secrets.HARBOR_USERNAME }}
        password: ${{ secrets.HARBOR_PASSWORD }}

    - name: Extract metadata
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
        tags: |
          type=ref,event=branch
          type=ref,event=pr
          type=semver,pattern={{version}}
          type=sha

    - name: Build and push
      id: build
      uses: docker/build-push-action@v5
      with:
        context: .
        push: ${{ github.event_name != 'pull_request' }}
        tags: ${{ steps.meta.outputs.tags }}
        labels: ${{ steps.meta.outputs.labels }}
        cache-from: type=registry,ref=${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:buildcache
        cache-to: type=registry,ref=${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:buildcache,mode=max

    - name: Scan image with Trivy
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
        format: 'sarif'
        output: 'trivy-results.sarif'
        severity: 'CRITICAL,HIGH'

deploy-dev:
  needs: build
  if: github.ref == 'refs/heads/develop'
  runs-on: ubuntu-latest
  environment: development
  steps:
    - uses: actions/checkout@v4
      with:
        repository: oilgas/platform-gitops
        token: ${{ secrets.GITOPS_TOKEN }}

    - name: Update image tag
      run: |
        cd charts/api
        yq -i '.image.tag = "${{ github.sha }}"' values-dev.yaml
        git config user.name github-actions
        git config user.email github-actions@github.com
        git commit -am "chore(api): update dev image to ${{ github.sha }}"
        git push

deploy-staging:
```

```
    needs: build
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
        with:
          repository: oilgas/platform-gitops
          token: ${{ secrets.GITOPS_TOKEN }}

      - name: Update image tag
        run: |
          cd charts/api
          yq -i '.image.tag = "${{ github.sha }}"' values-staging.yaml
          git config user.name github-actions
          git config user.email github-actions@github.com
          git commit -am "chore(api): update staging image to ${{ github.sha }}"
          git push

  deploy-production:
    needs: build
    if: startsWith(github.ref, 'refs/tags/v')
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
        with:
          repository: oilgas/platform-gitops
          token: ${{ secrets.GITOPS_TOKEN }}

      - name: Update image tag
        run: |
          VERSION=${GITHUB_REF#refs/tags/}
          cd charts/api
          yq -i '.image.tag = "'$VERSION'"' values-production.yaml
          git config user.name github-actions
          git config user.email github-actions@github.com
          git commit -am "chore(api): release $VERSION to production"
          git push
```

### 4.2 ML Model CI/CD

```
# .github/workflows/ml-pipeline.yml
name: ML Model Pipeline

on:
  push:
    paths:
      - 'ml-models/**'
    branches: [main]

jobs:
  train:
    runs-on: [self-hosted, gpu]
```

```yaml
    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install -r ml-models/requirements.txt

      - name: Train model
        env:
          MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_URI }}
        run: |
          cd ml-models/anomaly_detection
          python src/train.py --config configs/training_config.yaml

      - name: Evaluate model
        run: |
          cd ml-models/anomaly_detection
          python src/evaluate.py --model-uri models:/anomaly-detector/latest

  validate:
    needs: train
    runs-on: ubuntu-latest
    steps:
      - name: Run validation tests
        run: |
          pytest ml-models/tests/ -v

      - name: Check model metrics
        run: |
          # Fetch metrics from MLflow
          ACCURACY=$(mlflow runs get -r $RUN_ID --format json | jq '.metrics.accuracy')
          if (( $(echo "$ACCURACY < 0.95" | bc -l) )); then
            echo "Model accuracy $ACCURACY below threshold 0.95"
            exit 1
          fi

  deploy-model:
    needs: validate
    runs-on: ubuntu-latest
    steps:
      - name: Promote model to Production
        run: |
          mlflow models transition-stage \
            --name anomaly-detector \
            --version $MODEL_VERSION \
            --stage Production

      - name: Update edge deployment
        run: |
          # Trigger edge model update
```

```
curl -X POST $EDGE_MANAGER_URL/api/v1/models/deploy \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"model": "anomaly-detector", "version": "'$MODEL_VERSION'"}'
```

---

# 5. CONTAINER REGISTRY

## 5.1 Harbor Installation

```
# harbor-values.yaml
expose:
  type: ingress
  tls:
    enabled: true
    certSource: secret
    secret:
      secretName: harbor-tls
  ingress:
    hosts:
      core: harbor.oilgas.internal
    annotations:
      kubernetes.io/ingress.class: nginx
      nginx.ingress.kubernetes.io/ssl-redirect: "true"
      nginx.ingress.kubernetes.io/proxy-body-size: "0"

externalURL: https://harbor.oilgas.internal

persistence:
  enabled: true
  persistentVolumeClaim:
    registry:
      storageClass: ceph-rbd
      size: 500Gi
    database:
      storageClass: ceph-rbd
      size: 50Gi
    redis:
      storageClass: ceph-rbd
      size: 10Gi

database:
  type: external
  external:
    host: postgres.oilgas.internal
    port: 5432
    username: harbor
    password: ""
    sslmode: require

redis:
  type: internal

trivy:
```

```
  enabled: true
  gitHubToken: ""

notary:
  enabled: true

metrics:
  enabled: true
  serviceMonitor:
    enabled: true
```

```
helm repo add harbor https://helm.goharbor.io
helm install harbor harbor/harbor \
  --namespace harbor \
  --create-namespace \
  -f harbor-values.yaml
```

### 5.2 Image Policies

```
# image-policy.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: harbor-image-policy
data:
  policy.json: |
    {
      "rules": [
        {
          "name": "require-signed",
          "description": "Require images to be signed",
          "action": "block",
          "match": {
            "resources": {
              "repositories": ["oilgas/*"]
            }
          },
          "conditions": {
            "signature": {
              "required": true
            }
          }
        },
        {
          "name": "scan-before-deploy",
          "description": "Block images with critical vulnerabilities",
          "action": "block",
          "match": {
            "resources": {
              "repositories": ["oilgas/*"]
            }
          },
          "conditions": {
            "vulnerability": {
              "severity": "Critical",
```

```
            "count": 0
          }
        }
      }
    ]
  }
```

---

# 6. HELM CHARTS

## 6.1 API Chart Structure

```
charts/api/
  Chart.yaml
  values.yaml
  values-dev.yaml
  values-staging.yaml
  values-production.yaml
  templates/
      _helpers.tpl
      deployment.yaml
      service.yaml
      ingress.yaml
      hpa.yaml
      pdb.yaml
      configmap.yaml
      secret.yaml
      serviceaccount.yaml
      tests/
          test-connection.yaml
  charts/           # Dependencies
```

### Chart.yaml

```yaml
apiVersion: v2
name: oilgas-api
description: Oil & Gas AI Monitoring API
type: application
version: 1.0.0
appVersion: "1.0.0"

dependencies:
  - name: redis
    version: 18.x.x
    repository: https://charts.bitnami.com/bitnami
    condition: redis.enabled
```

### values.yaml

```yaml
# Default values
replicaCount: 3

image:
  repository: harbor.oilgas.internal/oilgas/api
```

```yaml
  pullPolicy: IfNotPresent
  tag: "latest"

imagePullSecrets:
  - name: harbor-pull-secret

serviceAccount:
  create: true
  annotations:
    iam.amazonaws.com/role: oilgas-api-role

podSecurityContext:
  runAsNonRoot: true
  runAsUser: 1000
  fsGroup: 1000

securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop:
      - ALL

service:
  type: ClusterIP
  port: 8000

ingress:
  enabled: true
  className: nginx
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/rate-limit: "100"
  hosts:
    - host: api.oilgas.internal
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: api-tls
      hosts:
        - api.oilgas.internal

resources:
  limits:
    cpu: 2000m
    memory: 4Gi
  requests:
    cpu: 500m
    memory: 1Gi

autoscaling:
  enabled: true
  minReplicas: 3
```

```yaml
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70
  targetMemoryUtilizationPercentage: 80

nodeSelector:
  node-role.kubernetes.io/worker: ""

tolerations: []

affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: oilgas-api
          topologyKey: kubernetes.io/hostname

env:
  - name: ENVIRONMENT
    value: "production"
  - name: LOG_LEVEL
    value: "info"

envFrom:
  - secretRef:
      name: api-secrets
  - configMapRef:
      name: api-config

livenessProbe:
  httpGet:
    path: /health/live
    port: http
  initialDelaySeconds: 10
  periodSeconds: 10
  failureThreshold: 3

readinessProbe:
  httpGet:
    path: /health/ready
    port: http
  initialDelaySeconds: 5
  periodSeconds: 5
  failureThreshold: 3

podDisruptionBudget:
  enabled: true
  minAvailable: 2

redis:
  enabled: false  # Use external Redis
```

**Deployment Template**

```yaml
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "api.fullname" . }}
  labels:
    {{- include "api.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "api.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
        checksum/secret: {{ include (print $.Template.BasePath "/secret.yaml") . | sha256sum }}
      labels:
        {{- include "api.selectorLabels" . | nindent 8 }}
    spec:
      {{- with .Values.imagePullSecrets }}
      imagePullSecrets:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      serviceAccountName: {{ include "api.serviceAccountName" . }}
      securityContext:
        {{- toYaml .Values.podSecurityContext | nindent 8 }}
      containers:
        - name: {{ .Chart.Name }}
          securityContext:
            {{- toYaml .Values.securityContext | nindent 12 }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - name: http
              containerPort: 8000
              protocol: TCP
          env:
            {{- toYaml .Values.env | nindent 12 }}
          envFrom:
            {{- toYaml .Values.envFrom | nindent 12 }}
          livenessProbe:
            {{- toYaml .Values.livenessProbe | nindent 12 }}
          readinessProbe:
            {{- toYaml .Values.readinessProbe | nindent 12 }}
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
          volumeMounts:
            - name: tmp
              mountPath: /tmp
```

```
      volumes:
        - name: tmp
          emptyDir: {}
      {{- with .Values.nodeSelector }}
      nodeSelector:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      {{- with .Values.affinity }}
      affinity:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      {{- with .Values.tolerations }}
      tolerations:
        {{- toYaml . | nindent 8 }}
      {{- end }}
```

---

# 7. SECRETS MANAGEMENT

## 7.1 Vault Setup

```
# Install Vault via Helm
helm repo add hashicorp https://helm.releases.hashicorp.com
helm install vault hashicorp/vault \
  --namespace vault \
  --create-namespace \
  --set server.ha.enabled=true \
  --set server.ha.replicas=3 \
  --set server.ha.raft.enabled=true

# Initialize Vault
kubectl exec -n vault vault-0 -- vault operator init -key-shares=5 -key-threshold=3

# Unseal (run on each replica)
kubectl exec -n vault vault-0 -- vault operator unseal <key1>
kubectl exec -n vault vault-0 -- vault operator unseal <key2>
kubectl exec -n vault vault-0 -- vault operator unseal <key3>
```

## 7.2 Vault Configuration

```
# Enable Kubernetes auth
vault auth enable kubernetes

vault write auth/kubernetes/config \
  kubernetes_host="https://$KUBERNETES_PORT_443_TCP_ADDR:443" \
  token_reviewer_jwt="$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)" \
  kubernetes_ca_cert=@/var/run/secrets/kubernetes.io/serviceaccount/ca.crt

# Create policy for API
vault policy write oilgas-api - <<EOF
path "secret/data/oilgas/api/*" {
  capabilities = ["read"]
}
```

```
path "secret/data/oilgas/database/*" {
  capabilities = ["read"]
}
EOF

# Create role for API
vault write auth/kubernetes/role/oilgas-api \
  bound_service_account_names=oilgas-api \
  bound_service_account_namespaces=oilgas-prod \
  policies=oilgas-api \
  ttl=1h
```

### 7.3 External Secrets Operator

```yaml
# external-secrets.yaml
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: vault-backend
  namespace: oilgas-prod
spec:
  provider:
    vault:
      server: "http://vault.vault.svc:8200"
      path: "secret"
      version: "v2"
      auth:
        kubernetes:
          mountPath: "kubernetes"
          role: "oilgas-api"
---
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: api-secrets
  namespace: oilgas-prod
spec:
  refreshInterval: 1h
  secretStoreRef:
    kind: SecretStore
    name: vault-backend
  target:
    name: api-secrets
    creationPolicy: Owner
  data:
    - secretKey: DATABASE_URL
      remoteRef:
        key: secret/data/oilgas/api/database
        property: url
    - secretKey: JWT_SECRET
      remoteRef:
        key: secret/data/oilgas/api/jwt
        property: secret
    - secretKey: INFLUXDB_TOKEN
```

```
      remoteRef:
        key: secret/data/oilgas/api/influxdb
        property: token
```

---

# 8. MONITORING STACK

## 8.1 Prometheus Stack

```
# prometheus-values.yaml
prometheus:
  prometheusSpec:
    retention: 30d
    storageSpec:
      volumeClaimTemplate:
        spec:
          storageClassName: ceph-rbd
          resources:
            requests:
              storage: 200Gi
    resources:
      requests:
        memory: 4Gi
        cpu: 2
      limits:
        memory: 8Gi
        cpu: 4
    additionalScrapeConfigs:
      - job_name: 'edge-nodes'
        static_configs:
          - targets:
            - 'edge-refa-01:9100'
            - 'edge-refa-02:9100'
            - 'edge-refb-01:9100'

alertmanager:
  config:
    global:
      resolve_timeout: 5m
    route:
      group_by: ['alertname', 'severity']
      group_wait: 30s
      group_interval: 5m
      repeat_interval: 12h
      receiver: 'pagerduty-critical'
      routes:
        - match:
            severity: critical
          receiver: 'pagerduty-critical'
        - match:
            severity: warning
          receiver: 'slack-warning'
    receivers:
```

```
      - name: 'pagerduty-critical'
        pagerduty_configs:
          - service_key: '<key>'
            severity: critical
      - name: 'slack-warning'
        slack_configs:
          - api_url: '<webhook>'
            channel: '#oilgas-alerts'

grafana:
  adminPassword: '<from-vault>'
  persistence:
    enabled: true
    storageClassName: ceph-rbd
    size: 10Gi
  dashboardProviders:
    dashboardproviders.yaml:
      apiVersion: 1
      providers:
        - name: 'oilgas'
          folder: 'Oil & Gas'
          type: file
          options:
            path: /var/lib/grafana/dashboards/oilgas
```

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install prometheus prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  -f prometheus-values.yaml
```

### 8.2 Custom Alerts

```
# alerts/oilgas-alerts.yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: oilgas-alerts
  namespace: monitoring
spec:
  groups:
    - name: oilgas.api
      rules:
        - alert: APIHighLatency
          expr: histogram_quantile(0.95, rate(http_request_duration_seconds_bucket{job="oilgas-api"}[5m]
          for: 5m
          labels:
            severity: warning
          annotations:
            summary: "High API latency detected"
            description: "P95 latency is {{ $value }}s"

        - alert: APIErrorRate
          expr: rate(http_requests_total{job="oilgas-api",status=~"5.."}[5m]) / rate(http_requests_total
          for: 2m
```

```
        labels:
          severity: critical
        annotations:
          summary: "High API error rate"
          description: "Error rate is {{ $value | humanizePercentage }}"

    - name: oilgas.ml
      rules:
        - alert: ModelInferenceLatency
          expr: histogram_quantile(0.99, rate(model_inference_duration_seconds_bucket[5m])) > 0.1
          for: 5m
          labels:
            severity: warning
          annotations:
            summary: "ML inference latency high"

        - alert: ModelAccuracyDrift
          expr: model_accuracy < 0.90
          for: 30m
          labels:
            severity: warning
          annotations:
            summary: "Model accuracy below threshold"

    - name: oilgas.edge
      rules:
        - alert: EdgeNodeOffline
          expr: up{job="edge-nodes"} == 0
          for: 5m
          labels:
            severity: critical
          annotations:
            summary: "Edge node {{ $labels.instance }} is offline"

        - alert: EdgeHighCPU
          expr: node_cpu_seconds_total{job="edge-nodes"} > 90
          for: 10m
          labels:
            severity: warning
          annotations:
            summary: "Edge node high CPU"
```

---

# 9. LOGGING STACK

## 9.1 ELK Stack Setup

```
# elasticsearch-values.yaml
clusterName: oilgas-logs
replicas: 3

esConfig:
  elasticsearch.yml: |
```

```
    cluster.name: "oilgas-logs"
    network.host: 0.0.0.0
    xpack.security.enabled: true
    xpack.security.transport.ssl.enabled: true

volumeClaimTemplate:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 500Gi
  storageClassName: ceph-rbd

resources:
  requests:
    cpu: 2
    memory: 8Gi
  limits:
    cpu: 4
    memory: 16Gi
```

```
# fluentbit-values.yaml
config:
  inputs: |
    [INPUT]
        Name              tail
        Tag               kube.*
        Path              /var/log/containers/*.log
        Parser            docker
        DB                /var/log/flb_kube.db
        Mem_Buf_Limit     50MB
        Skip_Long_Lines   On
        Refresh_Interval  10

  filters: |
    [FILTER]
        Name                kubernetes
        Match               kube.*
        Kube_URL            https://kubernetes.default.svc:443
        Kube_CA_File        /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
        Kube_Token_File     /var/run/secrets/kubernetes.io/serviceaccount/token
        Merge_Log           On
        K8S-Logging.Parser  On
        K8S-Logging.Exclude Off

  outputs: |
    [OUTPUT]
        Name            es
        Match           kube.*
        Host            elasticsearch-master
        Port            9200
        Logstash_Format On
        Logstash_Prefix oilgas-logs
        Retry_Limit     False
        tls             On
```

```
    tls.verify      Off
    HTTP_User       elastic
    HTTP_Passwd     ${ELASTIC_PASSWORD}
```

---

# 10. DISASTER RECOVERY

## 10.1 Backup Strategy

| Component | Method | Frequency | Retention | Location |
|---|---|---|---|---|
| etcd | Velero snapshot | Hourly | 30 days | S3 + DR site |
| PostgreSQL | pg_dump | Hourly | 30 days | S3 + DR site |
| InfluxDB | Continuous | Real-time | 7 years | DR site |
| Milvus | Daily export | Daily | 30 days | S3 |
| PVCs | Velero | Daily | 14 days | S3 |
| Git repos | Mirror | Real-time | Forever | DR site |

## 10.2 Velero Configuration

```yaml
# velero-values.yaml
configuration:
  provider: aws
  backupStorageLocation:
    name: default
    bucket: oilgas-velero-backups
    config:
      region: us-east-1
      s3ForcePathStyle: true
      s3Url: https://minio.oilgas.internal

  volumeSnapshotLocation:
    name: default
    config:
      region: us-east-1

credentials:
  secretContents:
    cloud: |
      [default]
      aws_access_key_id=<access-key>
      aws_secret_access_key=<secret-key>

schedules:
  daily-backup:
    schedule: "0 2 * * *"
    template:
      includedNamespaces:
        - oilgas-prod
      excludedResources:
        - pods
        - events
```

```
      snapshotVolumes: true
      ttl: 336h  # 14 days

# Manual backup
velero backup create pre-upgrade-backup \
  --include-namespaces oilgas-prod \
  --snapshot-volumes

# Restore
velero restore create --from-backup pre-upgrade-backup
```

### 10.3 DR Failover Procedure

```
DISASTER RECOVERY FAILOVER RUNBOOK
==================================

1. ASSESSMENT (5 min)
   - Confirm primary site is unavailable
   - Check DR site readiness
   - Notify stakeholders

2. DNS FAILOVER (10 min)
   - Update DNS to point to DR site
   - api.oilgas.com -> DR-IP
   - Verify propagation: dig api.oilgas.com

3. DATABASE FAILOVER (30 min)
   - Promote PostgreSQL replica to primary
   - Update connection strings
   - Verify data consistency

4. APPLICATION STARTUP (15 min)
   - Scale up DR deployments
   - Verify health checks
   - Run smoke tests

5. EDGE RECONNECTION (30 min)
   - Update edge VPN endpoints
   - Verify edge connectivity
   - Check data flow

6. VERIFICATION (30 min)
   - Run integration tests
   - Verify critical workflows
   - Check monitoring

7. NOTIFICATION (5 min)
   - Update status page
   - Notify stakeholders
   - Document timeline

TOTAL RTO: 2 hours
RPO: 1 hour (based on replication lag)
```

# 11. EDGE DEPLOYMENT

## 11.1 Edge GitOps with Fleet

```
# fleet.yaml
defaultNamespace: oilgas-edge

helm:
  repo: https://harbor.oilgas.internal/chartrepo/oilgas
  chart: edge-inference
  version: 1.0.0
  releaseName: edge-inference
  values:
    image:
      repository: harbor.oilgas.internal/oilgas/edge-inference
      tag: "1.0.0"
    resources:
      limits:
        nvidia.com/gpu: 1

targetCustomizations:
  - name: refinery-a
    clusterSelector:
      matchLabels:
        site: refinery-a
    helm:
      values:
        site: refinery-a
        kafka:
          brokers: kafka-refa.oilgas.internal:9092

  - name: offshore
    clusterSelector:
      matchLabels:
        site: offshore
    helm:
      values:
        site: offshore
        resources:
          requests:
            memory: 4Gi
```

## 11.2 Edge Update Script

```
#!/bin/bash
# edge-update.sh

set -e

EDGE_NODES=("edge-refa-01" "edge-refa-02" "edge-refb-01" "edge-offa-01")
NEW_VERSION=$1

if [ -z "$NEW_VERSION" ]; then
    echo "Usage: $0 <version>"
    exit 1
```

```
fi

for node in "${EDGE_NODES[@]}"; do
    echo "Updating $node to version $NEW_VERSION..."

    # Check connectivity
    if ! ssh edge@$node "echo 'Connected'" 2>/dev/null; then
        echo "WARNING: Cannot connect to $node, skipping"
        continue
    fi

    # Stop services
    ssh edge@$node "sudo systemctl stop inference-service"

    # Pull new image
    ssh edge@$node "docker pull harbor.oilgas.internal/oilgas/edge-inference:$NEW_VERSION"

    # Update model if needed
    ssh edge@$node "curl -O https://mlflow.oilgas.internal/models/anomaly-detector/$NEW_VERSION/model.e

    # Start services
    ssh edge@$node "sudo systemctl start inference-service"

    # Verify health
    sleep 10
    if ssh edge@$node "curl -s localhost:8080/health | grep -q healthy"; then
        echo "SUCCESS: $node updated to $NEW_VERSION"
    else
        echo "ERROR: $node health check failed"
        exit 1
    fi

    # Wait before next node
    sleep 30
done

echo "All nodes updated successfully!"
```

---

# 12. RUNBOOKS

## 12.1 Common Operations

### Scale API Deployment

```
# Manual scale
kubectl scale deployment oilgas-api -n oilgas-prod --replicas=10

# Update HPA limits
kubectl patch hpa oilgas-api -n oilgas-prod \
  --type='json' -p='[{"op": "replace", "path": "/spec/maxReplicas", "value": 20}]'
```

### Rolling Restart

```
# Restart all pods in deployment
kubectl rollout restart deployment oilgas-api -n oilgas-prod

# Watch progress
kubectl rollout status deployment oilgas-api -n oilgas-prod

# Rollback if needed
kubectl rollout undo deployment oilgas-api -n oilgas-prod
```

### Database Maintenance

```
# Run vacuum on PostgreSQL
kubectl exec -it postgres-0 -n databases -- \
  vacuumdb --all --analyze --verbose

# Check replication lag
kubectl exec -it postgres-0 -n databases -- \
  psql -c "SELECT client_addr, state, sent_lsn - replay_lsn as lag FROM pg_stat_replication;"
```

## 12.2 Incident Response

### High CPU Alert

```
RUNBOOK: High CPU Alert
=======================

1. Check current usage:
   kubectl top pods -n oilgas-prod --sort-by=cpu

2. Check recent deployments:
   kubectl get events -n oilgas-prod --sort-by='.lastTimestamp'

3. Scale if needed:
   kubectl scale deployment oilgas-api -n oilgas-prod --replicas=10

4. Check application logs:
   kubectl logs -l app=oilgas-api -n oilgas-prod --tail=100

5. If specific pod issue:
   kubectl delete pod <pod-name> -n oilgas-prod

6. If code issue:
   Rollback: kubectl rollout undo deployment oilgas-api -n oilgas-prod
```

### Database Connection Pool Exhausted

```
RUNBOOK: Database Connection Issues
===================================

1. Check connection count:
   kubectl exec -it postgres-0 -n databases -- \
     psql -c "SELECT count(*) FROM pg_stat_activity;"

2. Kill idle connections:
```

```
   kubectl exec -it postgres-0 -n databases -- \
     psql -c "SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE state = 'idle' AND query_sta
```

3. Increase max_connections (temporary):
```
   kubectl exec -it postgres-0 -n databases -- \
     psql -c "ALTER SYSTEM SET max_connections = 500;"
```

4. Scale API pods (reduce per-pod connections):
```
   kubectl scale deployment oilgas-api -n oilgas-prod --replicas=15
```

5. Investigate connection leaks in application logs

---

*END OF DEVOPS & DEPLOYMENT GUIDE*